Claude Code Docs

参考 › CLI 参考

参考

CLI 参考

复制页面

Claude Code 命令行界面的完整参考，包括命令和标志。

CLI 命令

您可以使用这些命令启动会话、管道内容、恢复对话和管理更新：

命令	描述	示例
`claude`	启动交互式会话	`claude`
`claude "query"`	使用初始提示启动交互式会话	`claude "explain this project"`
`claude -p "query"`	通过 SDK 查询，然后退出	`claude -p "explain this function"`
`cat file | claude -p "query"`	处理管道内容	`cat logs.txt | claude -p "explain"`
`claude -c`	继续当前目录中最近的对话	`claude -c`
`claude -c -p "query"`	通过 SDK 继续	`claude -c -p "Check for type errors"`
`claude -r "<session>" "query"`	按 ID 或名称恢复会话	`claude -r "auth-refactor" "Finish this PR"`
`claude update`	更新到最新版本	`claude update`
`claude auth login`	登录您的 Anthropic 账户。使用 `--email` 预填充您的电子邮件地址，使用 `--sso` 强制 SSO 身份验证	`claude auth login --email user@example.com --sso`
`claude auth logout`	从您的 Anthropic 账户登出	`claude auth logout`
`claude auth status`	以 JSON 格式显示身份验证状态。使用 `--text` 获取人类可读的输出。如果已登录，退出代码为 0，如果未登录，退出代码为 1	`claude auth status`

Claude Code Docs

›

`claude remote-control`	启动 **Remote Control 会话**，以在本地运行时从 Claude.ai 或 Claude 应用控制 Claude Code。请参阅 **Remote Control** 了解标志	`claude remote-control`

CLI标志

使用这些命令行标志自定义 Claude Code 的行为：

标志	描述	示例
`--add-dir`	添加额外的工作目录供 Claude 访问（验证每个路径是否存在为目录）	`claude --add-dir ../apps ../lib`
`--agent`	为当前会话指定代理（覆盖 `agent` 设置）	`claude --agent my-custom-agent`
`--agents`	通过 JSON 动态定义自定义 **subagents**（请参阅下面的格式）	`claude --agents '{"reviewer": {"description":"Reviews code","prompt":"You are a code reviewer"}}'`
`--allow-dangerously-skip-permissions`	启用权限绕过选项而不立即激活它。允许与 `--permission-mode` 组合（谨慎使用）	`claude --permission-mode plan --allow-dangerously-skip-permissions`
`--allowedTools`	无需提示权限即可执行的工具。请参阅 **权限规则语法** 了解模式匹配。要限制可用的工具，请改用 `--tools`	`"Bash(git log *)" "Bash(git diff *)" "Read"`
`--append-system-prompt`	将自定义文本附加到默认系统提示的末尾（在交互和打印模式中都有效）	`claude --append-system-prompt "Always use TypeScript"`
`--append-system-prompt-file`	从文件加载额外的系统提示文本并附加到默认提示（仅打印模式）	`claude -p --append-system-prompt-file ./extra-rules.txt "query"`
`--betas`	要包含在 API 请求中的 Beta 标头（仅限 API 密钥用户）	`claude --betas interleaved-thinking`
`--chrome`	启用 **Chrome 浏览器集成** 以进行网络自动化和测试	`claude --chrome`

Claude Code Docs

›

`--debug`	启用调试模式，可选类别过滤（例如，`"api,hooks"` 或 `"!statsig,!file"`）	`claude --debug "api,mcp"`
`--disable-slash-commands`	为此会话禁用所有 skills 和 slash commands	`claude --disable-slash-commands`
`--disallowedTools`	从模型的上下文中删除的工具，无法使用	`"Bash(git log *)" "Bash(git diff *)" "Edit"`
`--fallback-model`	当默认模型过载时启用自动回退到指定模型（仅打印模式）	`claude -p --fallback-model sonnet "query"`
`--fork-session`	恢复时，创建新的会话 ID 而不是重用原始 ID（与 `--resume` 或 `--continue` 一起使用）	`claude --resume abc123 --fork-session`
`--from-pr`	恢复链接到特定 GitHub PR 的会话。接受 PR 号或 URL。通过 `gh pr create` 创建时会自动链接会话	`claude --from-pr 123`
`--ide`	如果恰好有一个有效的 IDE 可用，则在启动时自动连接到 IDE	`claude --ide`
`--init`	运行初始化 hooks 并启动交互模式	`claude --init`
`--init-only`	运行初始化 hooks 并退出（无交互式会话）	`claude --init-only`
`--include-partial-messages`	在输出中包含部分流事件（需要 `--print` 和 `--output-format=stream-json`）	`claude -p --output-format stream-json --include-partial-messages "query"`
`--input-format`	为打印模式指定输入格式（选项：`text`、`stream-json`）	`claude -p --output-format json --input-format stream-json`
`--json-schema`	在代理完成其工作流后获得与 JSON Schema 匹配的验证 JSON 输出（仅打印模式，请参阅 **结构化输出**）	`claude -p --json-schema '{"type":"object","properties":{...}}' "query"`
`--maintenance`	运行维护 hooks 并退出	`claude --maintenance`
`--max-budget-usd`	API 调用前停止的最大美元金额（仅打印模式）	`claude -p --max-budget-usd 5.00 "query"`
`--max-turns`	限制代理轮数（仅打印模式）。达到限制时退出并出错。默认无限制	`claude -p --max-turns 3 "query"`

Claude Code Docs

›

	名（ sonnet 或 opus ）或模型的完整名称	4-6
--no-chrome	为此会话禁用 **Chrome 浏览器集成**	claude --no-chrome
--no-session-persistence	禁用会话持久化，以便会话不会保存到磁盘且无法恢复（仅打印模式）	claude -p --no-session-persistence "query"
--output-format	为打印模式指定输出格式（选项： text 、 json 、 stream-json ）	claude -p "query" --output-format json
--permission-mode	以指定的 **权限模式** 开始	claude --permission-mode plan
--permission-prompt-tool	指定 MCP 工具以在非交互模式下处理权限提示	claude -p --permission-prompt-tool mcp_auth_tool "query"
--plugin-dir	仅为此会话从目录加载 plugins（可重复）	claude --plugin-dir ./my-plugins
--print , -p	打印响应而不进入交互模式（请参阅 **Agent SDK 文档** 了解程序化使用详情）	claude -p "query"
--remote	在 claude.ai 上创建新的 **网络会话**，带有提供的任务描述	claude --remote "Fix the login bug"
--resume , -r	按 ID 或名称恢复特定会话，或显示交互式选择器以选择会话	claude --resume auth-refactor
--session-id	为对话使用特定的会话 ID（必须是有效的 UUID）	claude --session-id "550e8400-e29b-41d4-a716-446655440000"
--setting-sources	要加载的设置源的逗号分隔列表（ user 、 project 、 local ）	claude --setting-sources user,project
--settings	设置 JSON 文件的路径或 JSON 字符串以加载其他设置	claude --settings ./settings.json
--strict-mcp-config	仅使用 --mcp-config 中的 MCP 服务器，忽略所有其他 MCP 配置	claude --strict-mcp-config --mcp-config ./mcp.json
--system-prompt	用自定义文本替换整个系统提示（在交互和打印模式中都有效）	claude --system-prompt "You are a Python expert"
--system-prompt-file	从文件加载系统提示，替换默认提示（仅打印模式）	claude -p --system-prompt-file ./custom-prompt.txt "query"
--teleport	在本地终端中恢复 **网络会话**	claude --teleport

Claude Code Docs

›

`--tools`	限制 Claude 可以使用的内置工具（在交互和打印模式中都有效）。使用 `""` 禁用所有，`"default"` 表示全部，或工具名称如 `"Bash,Edit,Read"`	`claude --tools "Bash,Edit,Read"`
`--verbose`	启用详细日志记录，显示完整的逐轮输出（有助于在打印和交互模式中调试）	`claude --verbose`
`--version` , `-v`	输出版本号	`claude -v`
`--worktree` , `-w`	在隔离的 **git worktree** 中启动 Claude，位于 `<repo>/.claude/worktrees/<name>` 。如果未给出名称，则自动生成一个	`claude -w feature-auth`

`--output-format json` 标志对于脚本和自动化特别有用，允许您以编程方式解析 Claude 的响应。

Agents 标志格式

`--agents` 标志接受一个 JSON 对象，该对象定义一个或多个自定义 subagents。每个 subagent 需要一个唯一的名称（作为键）和一个具有以下字段的定义对象：

字段	必需	描述
`description`	是	何时应调用 subagent 的自然语言描述
`prompt`	是	指导 subagent 行为的系统提示
`tools`	否	subagent 可以使用的特定工具数组，例如 `["Read", "Edit", "Bash"]` 。如果省略，继承所有工具。支持 `Task(agent_type)` 语法
`disallowedTools`	否	为此 subagent 明确拒绝的工具名称数组
`model`	否	要使用的模型别名：`sonnet` 、`opus` 、`haiku` 或 `inherit` 。如果省略，默认为 `inherit`
`skills`	否	要预加载到 subagent 上下文中的 **skill** 名称数组
`mcpServers`	否	此 subagent 的 **MCP servers** 数组。每个条目是服务器名称字符串或 `{name: config}` 对象

Claude Code Docs

›

```
claude --agents '{
  "code-reviewer": {
    "description": "Expert code reviewer. Use proactively after code changes.",
    "prompt": "You are a senior code reviewer. Focus on code quality, security,
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  },
  "debugger": {
    "description": "Debugging specialist for errors and test failures.",
    "prompt": "You are an expert debugger. Analyze errors, identify root causes
  }
}'
```

有关创建和使用 subagents 的更多详情，请参阅 。

系统提示标志

Claude Code 提供四个标志用于自定义系统提示，每个标志服务于不同的目的：

标志	行为	模式	用例
`--system-prompt`	**替换**整个默认提示	交互 + 打印	完全控制 Claude 的行为和指令
`--system-prompt-file`	**替换**为文件内容	仅打印	从文件加载提示以实现可重现性和版本控制
`--append-system-prompt`	**附加**到默认提示	交互 + 打印	添加特定指令，同时保持默认 Claude Code 行为
`--append-system-prompt-file`	**附加**文件内容到默认提示	仅打印	从文件加载额外指令，同时保持默认值

何时使用每个：

Claude Code Docs

›

`--system-prompt-file` ：当您想从文件加载自定义提示时使用，对于团队一致性或版本控制的提示模板很有用。

```
claude -p --system-prompt-file ./prompts/code-review.tx
```

`--append-system-prompt` ：当您想添加特定指令同时保持 Claude Code 的默认功能完整时使用。这是大多数用例的最安全选项。

```
claude --append-system-prompt "Always use TypeScript an
```

`--append-system-prompt-file` ：当您想从文件附加指令同时保持 Claude Code 的默认值时使用。对于版本控制的添加很有用。

```
claude -p --append-system-prompt-file ./prompts/style-r
```

`--system-prompt` 和 `--system-prompt-file` 互斥。附加标志可以与任一替换标志一起使用。

对于大多数用例，建议使用 `--append-system-prompt` 或 `--append-system-prompt-file` ，因为它们保留 Claude Code 的内置功能，同时添加您的自定义要求。仅当您需要完全控制系统提示时才使用 `--system-prompt` 或 `--system-prompt-file` 。

另请参阅

- 浏览器自动化和网络测试
- 快捷键、输入模式和交互功能
- Claude Code 入门

Claude Code Docs

此页面对您有帮助吗?

是 否

交互模式

Claude Code Docs

›

Careers

Economic Futures

Research

News

Trust center

Transparency

Status

Support center

Learn

Courses

MCP connectors

Customer stories

Engineering blog

Events

Powered by Claude

Service partners

Startups program

Terms and policies

Privacy policy

Disclosure policy

Usage policy

Commercial terms

Consumer terms